

Regole per la risoluzione degli alberi dei processi

1. `pid = fork();` -> Crea una copia esatta del processo padre, incluse le variabili. Nel processo padre, `fork()` restituisce il PID del figlio, che è un numero >0 (quindi `pid > 0`), mentre nel processo figlio `pid` assume il valore 0 (`pid == 0`).
2. `if((pid = fork()))` -> Crea una copia esatta del processo padre, incluse le variabili. Si comporta come `pid=fork()`, ma il corpo dell'`if` viene eseguito **solo dal processo padre**, poiché `fork()` restituisce in `pid` un valore >0 (true). Nel figlio, invece, `pid == 0`, quindi la condizione risulta falsa e il corpo non viene eseguito.
3. `if(!(pid = fork()))` -> Crea una copia esatta del processo padre, incluse le variabili. Si comporta come `pid=fork()` ma il corpo dell'`if` viene eseguito solo dal processo figlio, poiché nel figlio `fork()` restituisce 0, e negandolo `!0 = true`. Nel padre `pid > 0`, quindi `!pid = false`.
4. `if((pid = fork()) == 0)` -> Crea una copia esatta del processo padre, incluso le variabili. Il corpo dell'`if` viene eseguito **solo dal processo figlio**, poiché la condizione è vera solo quando `fork()` restituisce 0.
5. `fork();` -> Crea una copia esatta del processo padre, incluse le variabili. **NB:** In questo caso, non c'è assegnazione come in `pid = fork()`, quindi il valore restituito da `fork()` **non viene salvato** in nessuna variabile.
6. `if(fork())` -> Crea una copia esatta del processo padre, incluse le variabili. Nel corpo dell'`if` entra **solo il processo padre**, poiché `fork()` restituisce un valore > 0 (vero). Nei figli, `fork()` restituisce 0 (falso), quindi non eseguono il corpo dell'`if`. **NB:** Così scritta, questa istruzione **non salva alcun valore** in `pid`, né nel padre né nel figlio.
7. `if(!fork())` -> Crea una copia esatta del processo padre, incluse le variabili. Nel corpo dell'`if` entra **solo il processo figlio**, poiché `fork()` restituisce 0 nel figlio e `!0 = true`. Nel padre, `fork()` restituisce un numero >0 , quindi `!>0 = false`. **NB:** Così scritta, questa istruzione **non salva alcun valore** in `pid`, né nel padre né nel figlio.
8. `if(pid)` -> Il corpo dell'`if` viene eseguito **solo dai processi che hanno** `pid > 0`.
9. `if(!pid)` -> Il corpo dell'`if` viene eseguito **solo dai processi che hanno** `pid == 0` (cioè i figli).
10. `if(glob)` -> Il corpo dell'`if` viene eseguito **solo dai processi che hanno** `glob > 0`.
11. `if(!glob)` -> Il corpo dell'`if` viene eseguito **solo dai processi che hanno** `glob == 0`.
12. `while(glob)` -> Il corpo del ciclo viene eseguito **solo dai processi che hanno** `glob > 0`. Il ciclo termina quando `glob == 0`.
13. `while(!glob)` -> Il corpo del ciclo viene eseguito **solo dai processi che hanno** `glob == 0`. Il ciclo termina quando `glob > 0`.

14. **while(fork())** -> Crea una copia esatta del processo padre, incluse le variabili. Nel corpo del ciclo entra **solo il processo padre**, poiché `fork()` restituisce in quest'ultimo un valore > 0 . Il figlio, invece, riceve 0 da `fork()` e non entra nel ciclo. Il ciclo continua a creare nuovi figli fino a quando una condizione interna non lo interrompe. **NB:** Senza una condizione di uscita, si rischia una moltiplicazione esponenziale dei processi.
15. **while(!fork())** -> Crea una copia esatta del processo padre, incluse le variabili. Nel corpo del ciclo entra **solo il processo figlio**, poiché `fork()` restituisce 0 in quest'ultimo e `!0 = true`. Il padre riceve un valore > 0 e quindi `!>0 = false`. Il ciclo continua a creare nuovi figli fino a quando una condizione interna non lo interrompe. **NB:** Anche qui, senza una condizione di uscita, si può generare una catena infinita di processi.